Homework 11 Report — Album Store

Name: Weifan Li

Nickname: lhq5520

Final Score: 179 / 190

1. Roughly how many submissions did it take before you passed all critical scenarios, and what was the most common failure?

It took about 2 submissions to pass all critical scenarios (S1–S5). The first submission already passed all critical scenarios. The most common failure across all submissions was in load testing scenarios — specifically S14 (Mixed Metadata + Uploads) and S15 (Large Payload Upload), where concurrent S3 uploads caused either memory pressure or disk exhaustion.

2. Where are your photo files stored, and why did you pick that over other options?

Photos are stored in Amazon S3 in the same region (us-west-2) as the ChaosArena test runner. I chose S3 because it provides durable storage with publicly accessible URLs (required by the spec), and placing it in the same region minimizes network latency. Using a VPC Gateway Endpoint for S3 keeps traffic on AWS's internal network, avoiding the public internet entirely.

3. Describe your deployment setup — how many instances, what cloud services, and how they connect to each other.

Single EC2 instance (c6i.xlarge) running the Go binary directly on port 80, with an Elastic IP. It connects to an RDS PostgreSQL instance for metadata storage and S3 for photo files. All three resources are in us-west-2a (same AZ) to minimize inter-service latency. Infrastructure is managed with Terraform.

4. Did you use a reverse proxy or load balancer? If so, what role does it play in your architecture?

No. The Go HTTP server listens directly on port 80 with an Elastic IP. I intentionally avoided a load balancer or reverse proxy to eliminate an extra network hop, since every millisecond of latency affects the load test score.

5. How does your background worker get notified that there's a new photo to process? Did you use a queue, polling, or something else?

I use a Go goroutine launched directly from the POST handler — no external queue or polling. When a photo is uploaded, the handler reads the file, assigns a seq number, returns 202, and then spawns a goroutine that uploads to S3 and updates the database. A semaphore (buffered channel of size 20) limits concurrent uploads to control memory pressure. This avoids the 20–50ms overhead of an external message queue like SQS.

6. The spec requires that seq is assigned in the POST handler, not the background worker. Why does that matter, and how did you ensure correctness under concurrent uploads to the same album?

Assigning seq in the POST handler guarantees that the client receives a correct, unique sequence number in the 202 response immediately — before any async processing begins. If the worker assigned it, there would be a race condition where two concurrent uploads could get the same seq. I ensure correctness by using a PostgreSQL transaction: UPDATE albums SET next_seq = next_seq + 1 WHERE album_id = $1 RETURNING next_seq, followed by the photo INSERT, all within a single transaction. PostgreSQL's row-level locking on the UPDATE guarantees atomicity even under high concurrency.

7. What happens in your system if the worker crashes or fails halfway through processing a photo?

If the S3 upload fails, the goroutine catches the error and updates the photo status to "failed" in both the database and the in-memory cache. If the entire process crashes (e.g., OOM kill), the photo remains in "processing" state permanently. Systemd automatically restarts the service within 3 seconds, but orphaned "processing" records are not cleaned up — a proper production system would need a periodic reconciliation job.

8. What does your database schema look like? What tables or collections did you create and why?

Two tables: (1) albums — stores album_id (UUID PK), title, description, owner, and next_seq (integer counter for per-album photo sequence numbers). (2) photos — stores photo_id (UUID PK), album_id (FK to albums), seq, status (processing/completed/failed), url, and created_at.

The next_seq column on the albums table enables atomic sequence number allocation in a single UPDATE...RETURNING query, avoiding a separate counter table or sequence.

9. Did you add any indexes to your database? If so, on which columns and why?

Yes, two indexes beyond the primary keys: (1) idx_photos_album on photos(album_id) — speeds up queries that look up all photos in an album. (2) idx_photos_album_photo on photos(album_id, photo_id) — optimizes the GET and DELETE endpoints which filter by both album_id and photo_id, allowing index-only lookups.

10. Which load testing scenario was the hardest for you, and what bottleneck did you discover?

S14 (Mixed Metadata + Uploads) was the hardest. Initially, I used an unbounded number of goroutines for S3 uploads, which caused extreme GC pressure under concurrent load — the upload p95 reached 62 seconds. The bottleneck was Go's garbage collector scanning hundreds of large byte slices held by concurrent goroutines. Adding a semaphore to cap concurrent uploads at 20 immediately brought S14 from 0 points to 15 (full score).

11. What was the single most impactful change you made to improve your load test scores?

Adding a semaphore (buffered channel of size 20) to limit concurrent S3 uploads. This single change took S14 from 0/15 to 15/15 and improved the overall score from 163 to 179. It controlled memory usage and GC pressure without sacrificing throughput.

12. How did you handle concurrent writes — for example, many album creates or photo uploads happening at the same time?

For album creates: PostgreSQL's INSERT ... ON CONFLICT DO UPDATE (UPSERT) handles concurrent PUTs to the same album_id atomically. For photo uploads: the seq allocation uses UPDATE ... RETURNING inside a transaction with row-level locking, ensuring each concurrent upload gets a unique sequence number. The Go HTTP server handles concurrency natively with goroutines — no external synchronization needed.

13. Describe a specific bug you ran into and how you diagnosed it using the ChaosArena event logs or your own logs.

S9 (Delete Before Complete) failed with: "orphaned record: photo returned 200 after DELETE — async worker wrote metadata after deletion was confirmed". The event log showed that after deleting a photo, a GET request 2 seconds later returned 200 instead of 404. The root cause: the upload goroutine was still running after the DELETE, and when it completed the S3 upload, it wrote the photo back to both the database and the in-memory cache. I fixed it by adding WHERE status = 'processing' to the UPDATE query, so a deleted photo's row can never be resurrected, and only updating the cache when the DB update actually affected a row.

14. How did you test your service locally before submitting to ChaosArena?

I tested each endpoint with curl commands directly on the EC2 instance (localhost), verifying correct JSON responses and status codes. For example: curl http://localhost/health, curl -X PUT http://localhost/albums/<uuid>, etc. I also checked systemctl status and journalctl logs to ensure the service started correctly and the database connection was healthy.

15. If you had another week, what is the one thing you would change or add to your system to improve your score?

I would migrate from standard S3 to S3 Express One Zone, which offers single-digit millisecond latency — roughly 10x faster than standard S3. The remaining 11 points I'm missing (S12 and S15) are entirely bottlenecked by S3 upload latency, so this single change could potentially push the score close to 190.

16. How did you add value over and above what Claude could do in this assignment?

Claude cannot interact with AWS directly — it cannot create key pairs, run Terraform, SSH into EC2, or debug live deployment issues. I handled all the real-world infrastructure work: configuring AWS CLI credentials, running terraform apply, creating and managing SSH keys, fixing file permission issues on Windows, uploading binaries via SCP, diagnosing "address already in use" errors during service restarts, and interpreting live journalctl logs to identify issues like disk space exhaustion. I also made the iterative submit-and-tune decisions based on ChaosArena results, choosing which optimization direction to pursue at each step.